5 Virtual Memory

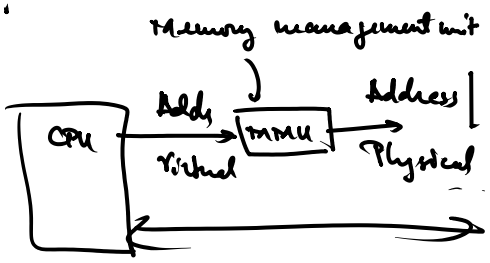
Tuesday, May 25, 2021 12:48 PM

5.1 Motivation

1. Efficient memory sharing between programs/processes (virtual machines)
2. Remove the burden of a limited amount of Main Memory

=> MM need contain just the active portions of the many processes / virtual machines
- according/abiding the locality principle!

Virtual Address —(Translation, mapping)→ Physical address



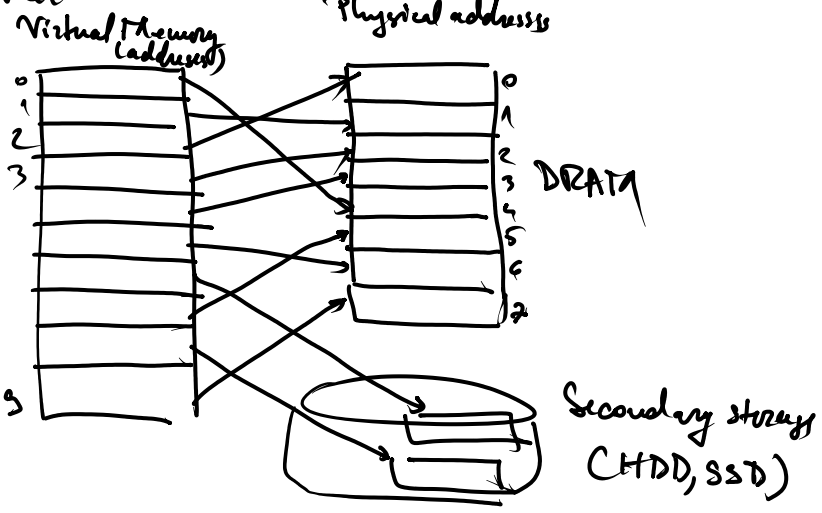
Translation entails protection for the program's memory space (virtual machine's) from other programs / virtual machines.

Historically → Overlay

VM blocks → page

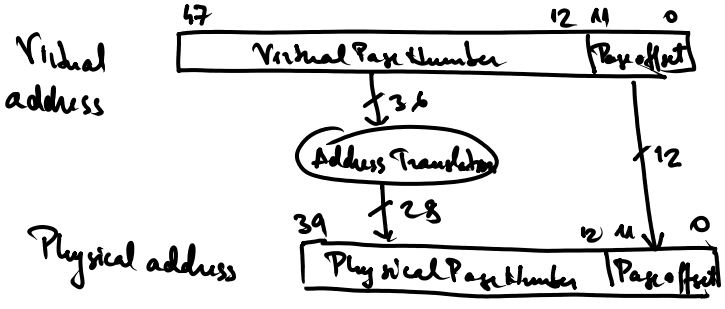
VM miss → page fault





Address mapping ARM V8

Address word 64-bits (first 16 bits are unused)



PM Size = 1 TiB

VM Size = 256 TiB

Page size = 2^{12} = 4 KiB

Design choices — motivated by a large miss penalty (page fault reduction)

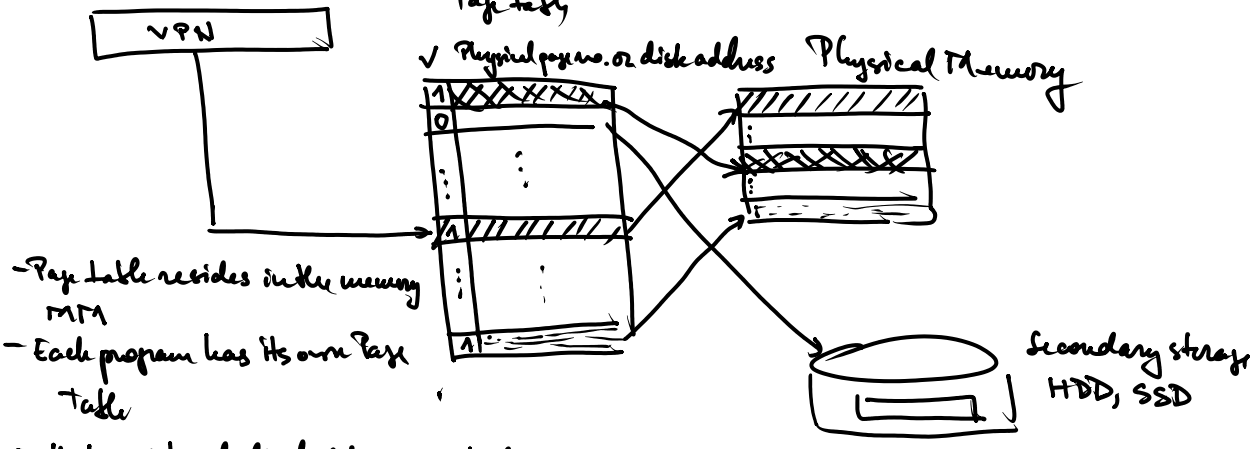
- at HDD - millions of c.c. in penalty

- pages should be large enough to amortize the large cost of secondary storage access time → typically 4 KiB - 64 KiB
  - desktops, servers → 32 KiB, 64 KiB
  - embedded → 1 KiB, 4 KiB
- Organizations that reduce fault/miss rate → Full Associative
  - page faults are handled in SW
  - more complex replacement algorithms in SW
- WT is unacceptable => only Write Back

ARM: page → granule, page fault → MMU exception

5.2 Page placement

Finding a page - in caches: Tags
- in Virtual Memory: Page Tables



- HW register dedicated to page tables → page table registers

For a process or Virtual Machine

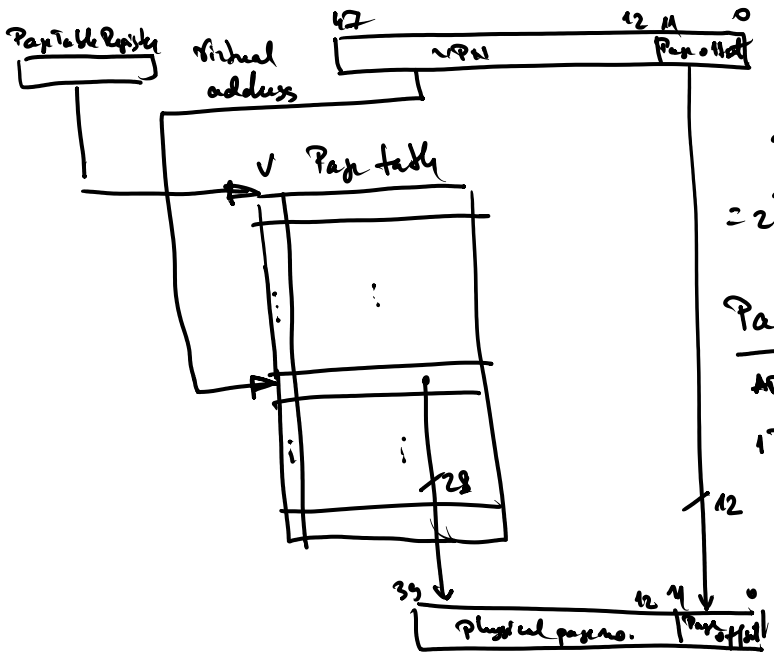
Page table + PC (program counter) + registers = the state of the process / virtual machine

- process / Virtual Machine - active = in possession of the CPU
  - inactive = not in possession of the CPU
- when the process is inactive, its status is saved

ARM V8

- when one process is inactive, its status is saved

ARM V8



2^{36} entries $\times \underbrace{32 \text{ bits}}_{2^2 B} =$

$= 2^{36} \times 2^2 B = 2^{38} B = 0.5 \text{ TiB}$

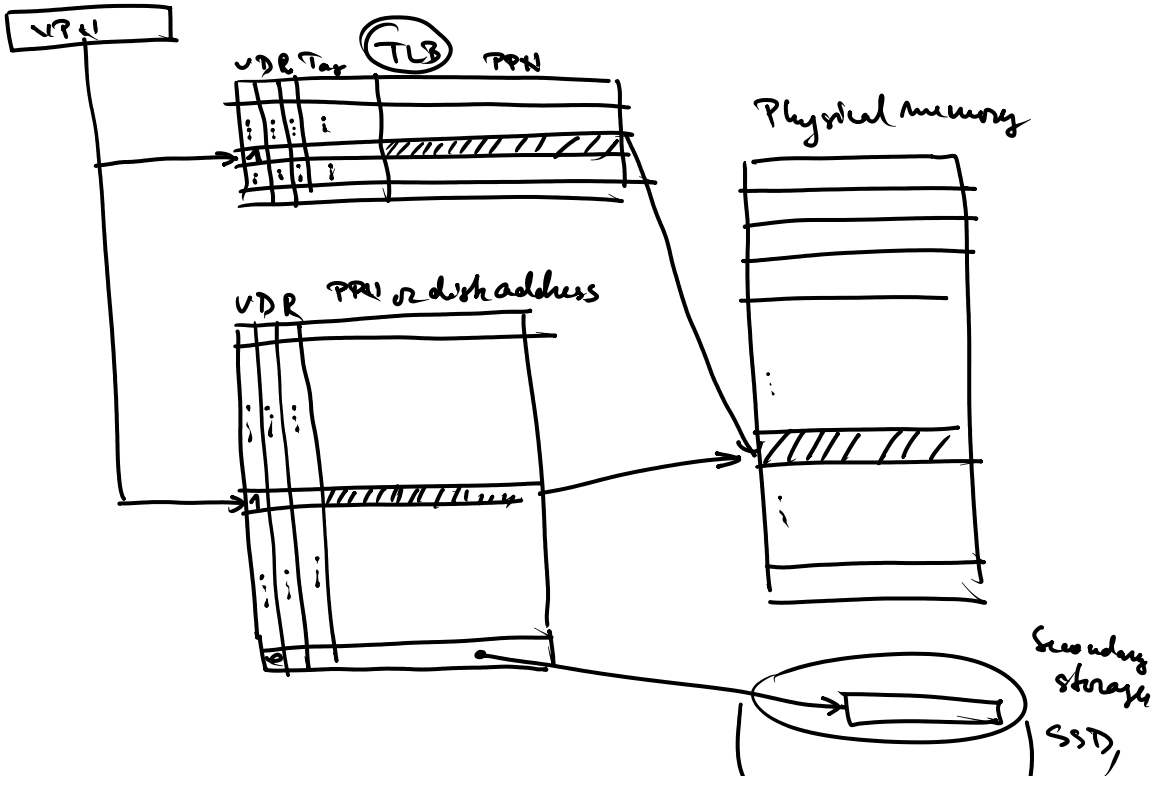
Page table size reduction

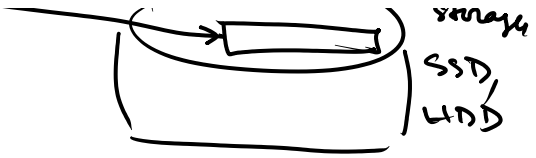
ARM V8 Multi-level, paged tables

1 Table = 4 pages (4 KiB)

Speeding the access time to Page Tables

- Caching the page table (TLB - Translation Lookaside Buffer)
  - FA, but can also be SA, DM
  - caches the Page Table entries





Intrinsity's FastMATH

TLB: 16 – 512 entries

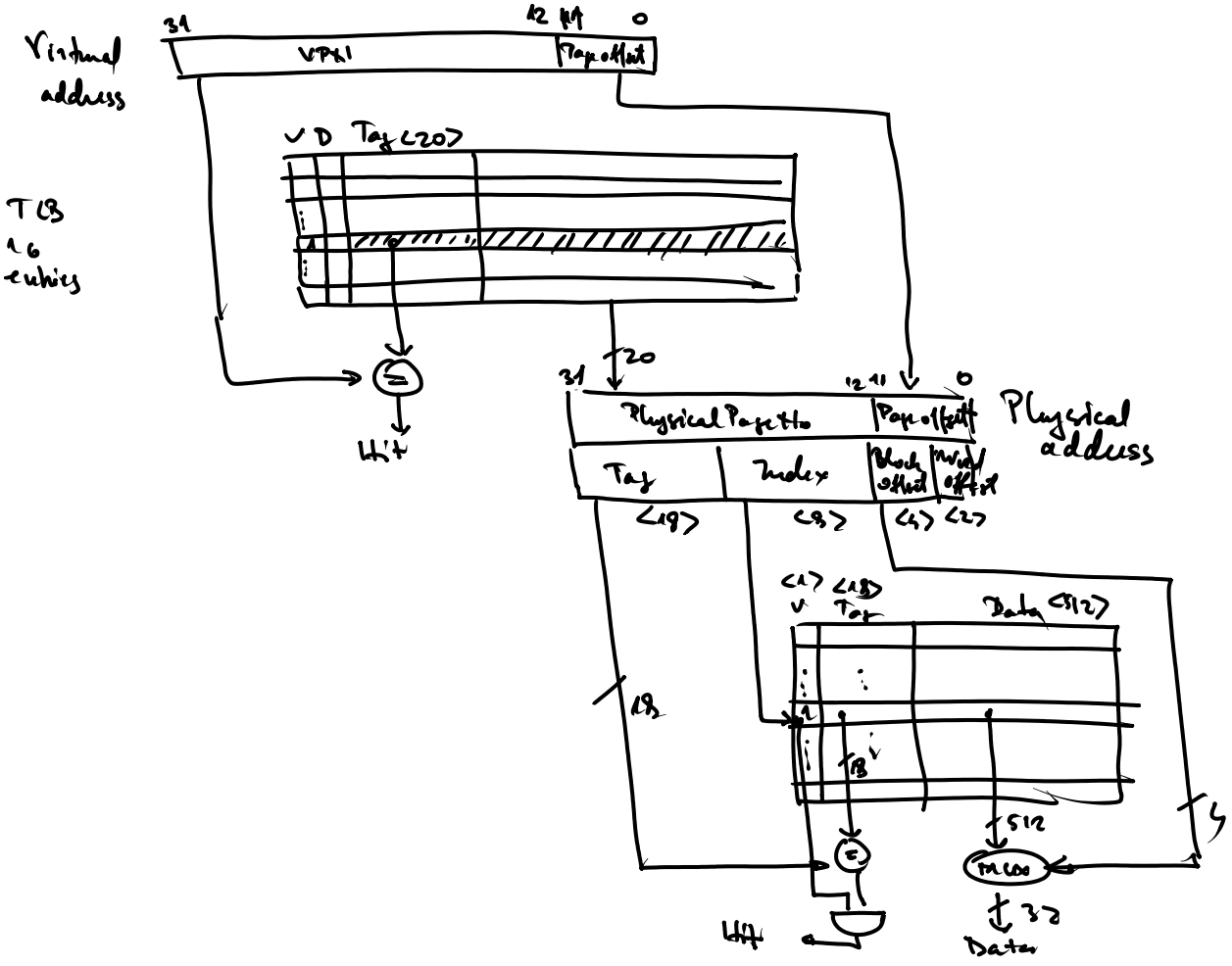
Block size: 1–2 Page Table entries

16 entries, FA TLB

Hit time = 1 c.c.

Miss Penalty – 10's – 100's c.c.

Miss Rate = 0.1 — 1%



Characteristic	ARM Cortex-A8	Intel Core i7
Virtual address	32 bits	48 bits
Physical address	32 bits	44 bits
Page size	Variable: 4, 16, 64 KiB, 1, 16 MiB	Variable: 4 KiB, 2/4 MiB
TLB organization	1 TLB for instructions and 1 TLB for data Both TLBs are fully associative, with 32 entries, round robin replacement TLB misses handled in hardware	1 TLB for instructions and 1 TLB for data per core Both L1 TLBs are four-way set associative, LRU replacement L1 I-TLB has 128 entries for small pages, 7 per thread for large pages L1 D-TLB has 64 entries for small pages, 32 for large pages The L2 TLB is four-way set associative, LRU replacement The L2 TLB has 512 entries TLB misses handled in hardware

Characteristic	ARM Cortex-A8	Intel Nehalem
L1 cache organization	Split instruction and data caches	Split instruction and data caches
L1 cache size	32 KiB each for instructions/data	32 KiB each for instructions/data per core
L1 cache associativity	4-way (I), 4-way (D) set associative	4-way (I), 8-way (D) set associative
L1 replacement	Random	Approximated LRU
L1 block size	64 bytes	64 bytes
L1 write policy	Write-back, Write-allocate(?)	Write-back, No-write-allocate
L1 hit time (load-use)	1 clock cycle	4 clock cycles, pipelined
L2 cache organization	Unified (instruction and data)	Unified (instruction and data) per core
L2 cache size	128 KiB to 1 MiB	256 KiB (0.25 MiB)
L2 cache associativity	8-way set associative	8-way set associative
L2 replacement	Random(?)	Approximated LRU
L2 block size	64 bytes	64 bytes
L2 write policy	Write-back, Write-allocate (?)	Write-back, Write-allocate
L2 hit time	11 clock cycles	10 clock cycles
L3 cache organization	–	Unified (instruction and data)
L3 cache size	–	8 MiB, shared
L3 cache associativity	–	16-way set associative
L3 replacement		Approximated LRU
L3 block size		64 bytes
L3 write policy	–	Write-back, Write-allocate
L3 hit time	–	35 clock cycles